

★ WEEK 8 · LECTURE

Neural Networks & Deep Learning

The neuron, layers, backprop — and the Autoencoder

Expanded edition · 38 slides · with three hands-on labs

The idea behind today's AI boom, built from one simple part: the neuron.
We stack them, teach them to learn from mistakes, and meet anomaly detector #4.

Where We Are Going Today

Week 7 was trees. Now we meet the networks behind modern AI — gently

1

One neuron (the perceptron)

A tiny decision-maker that weighs clues and fires. The building block of everything.

2

Stacking neurons into deep networks

Layers that build simple ideas into complex ones — and how they learn (backprop).

3

The Autoencoder

Anomaly detector #4 — squeeze data through a bottleneck and watch what won't rebuild.

By the end you will fire a neuron by hand, walk downhill, and complete the four-detector family.

1

PART 1 · ONE NEURON

One Neuron

The perceptron: the building block

How Do YOU Make a Decision?

A neuron copies the way you weigh things up

Deciding to go outside, you weigh: sunny? (a lot) have time? (a lot) cold? (counts against). Add up the pluses and minuses; if the total is high enough, you go. A neuron does exactly this.

The three parts

- Inputs (x): the clues.
- Weights (w): how much each clue matters.
- Activation: the 'fire or not' decision.

Why it's powerful

- One neuron is simple — almost too simple.
- But millions of them, stacked, can recognize faces and speech.
- Complexity emerges from many tiny parts.

Hype, Winter, and a Comeback

Neural nets were hyped in 1958, abandoned, then changed the world

**"In 1958 the perceptron made front-page news. A decade later critics buried it.
Forty years on, it powers nearly all modern AI."**

The 'AI winter'

- Early perceptrons couldn't solve simple problems alone.
- Funding dried up; the field went quiet for years.
- The missing pieces: more layers, more data, more compute.

The comeback

- Backpropagation made deep networks trainable.
- Big data + fast GPUs unlocked their power.
- Today: image, speech, language — and our chatbot (Week 12).

Sixty Years, Four Numbers

From a front-page machine to the engine of everything

1958

The perceptron makes
the New York Times
front page

1969

Critics show one
neuron can't even do
XOR — winter falls

~2012

GPUs + big data: deep
nets sweep vision,
then everything

#4

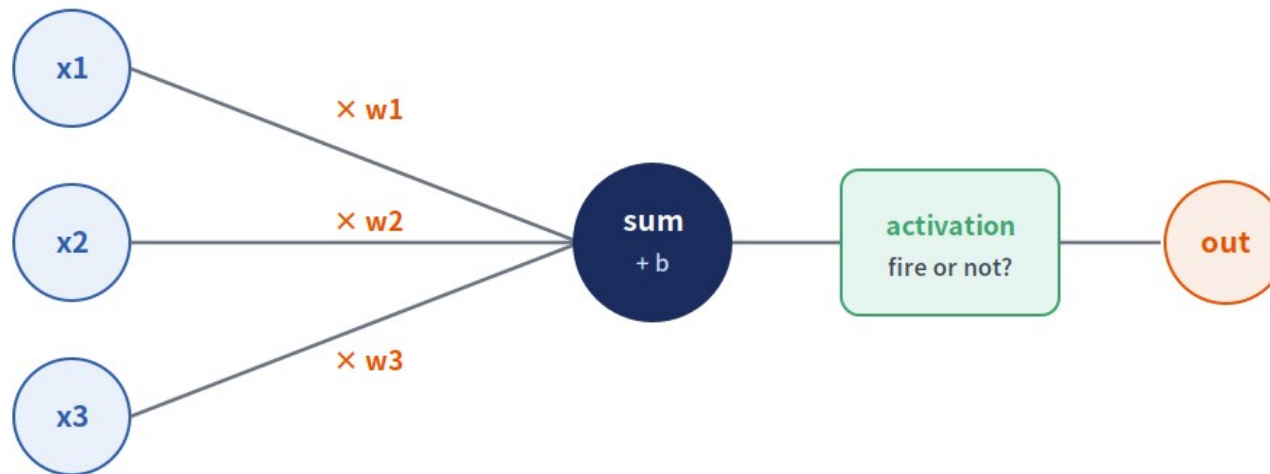
Today's cameo in OUR
course: the
Autoencoder detector

Keep the 1969 complaint in mind — we will solve XOR ourselves in a few slides.

Inside a Single Neuron

Weigh the inputs, add them up, then decide to fire — the Lego brick of every neural network.

A Neuron — Weigh the Clues, Then Decide to Fire



Like deciding to go out

Is it sunny? (x_1) Do I have time? (x_2) Is it cold? (x_3)

Give each a weight (how much it matters), add them up.

If the total is high enough, the neuron "fires" → go out!

A Neuron, as a Formula

A weighted sum, then a 'fire?' function

$$\text{output} = f(w_1x_1 + w_2x_2 + \dots + b)$$

Reading it

- Inside the brackets: the weighted sum (clues times weights, plus a bias b).
- f is the activation — the 'fire or not' function wrapped around it.
- It's the same linear score from Week 4, passed through a switch.

Look familiar?

- If f is the S-curve, this is exactly Logistic Regression (Week 5)!
- So one neuron = logistic regression.
- The magic comes from STACKING many of them.

Fire a Neuron by Hand

The Week 1 credit score IS a neuron: $w = (-1, +2)$, $b = 0$ (full version: problem set)

Applicant	(Income, Debt)	$z = 2 \cdot \text{Debt} - \text{Income}$	step(z): fire?	sigmoid(z)
A	(1.0, 0.8)	+0.6	1 — FIRE (risky)	0.65
B	(2.0, 0.5)	-1.0	0 — silent	0.27
G	(1.1, 0.5)	-0.1	0 — silent	0.47

Swap the step for a sigmoid and you get Week 5's exact probabilities — one neuron really is logistic regression.

Why We Need the 'Fire' Switch

Without it, a deep network is just one boring line

$$\text{ReLU}(z) = \max(0, z)$$

Without a switch

- Stacking straight lines just gives... another straight line.
- Ten layers would be no smarter than one.
- The network could only draw straight boundaries.

With a switch (non-linearity)

- Each layer can bend the picture; stacked bends create rich patterns.
- This is what lets deep nets learn complex shapes.
- ReLU — pass positives, flatten negatives — is simple and works astonishingly well.

2

PART 2 · STACKING

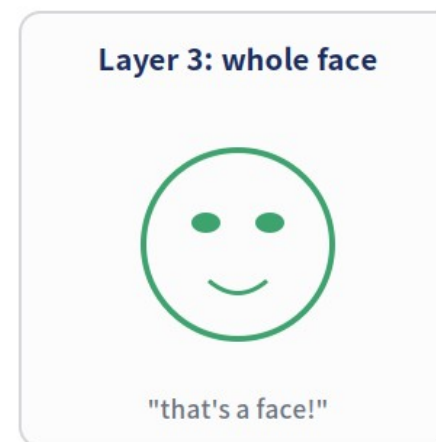
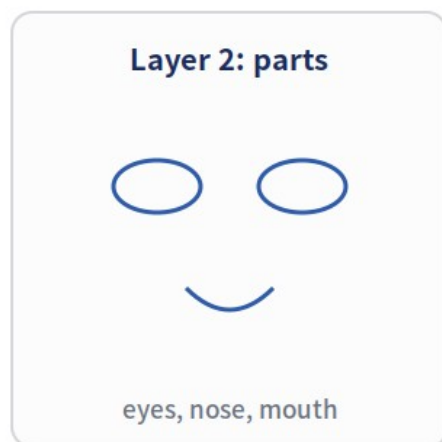
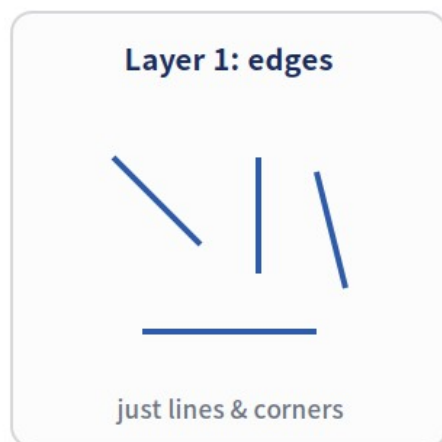
Stacking Neurons

Deep networks: simple to complex

Layers Build Ideas, Step by Step

Early layers see simple things; later layers combine them. 'Deep' just means many layers stacked.

Deep Network — Each Layer Builds a Bigger Idea



The big idea

Early layers catch simple patterns; later layers combine them into complex ones.

"Deep" = many layers stacked up.

What 'Deep Learning' Really Means

More layers = more power, but also more hunger

The strengths

- Learns features automatically — no hand-crafting.
- Captures very complex, non-linear patterns.
- Scales up with more data and compute; state-of-the-art for images, speech, language.

The costs (especially in finance)

- Needs LOTS of data — finance often has little.
- Hungry for compute and tuning; a black box (Week 14 helps).
- Easy to overfit noisy markets (Week 1).

For tabular finance data, Week 7's XGBoost often still wins. Deep nets shine on text & images.

Solving 1969's Complaint by Hand

XOR — 'fire on exactly one clue' — beats any single neuron. Two layers do it in four rows

(x_1, x_2)	$h_1 = \text{step}(x_1 + x_2 - 0.5)$ 'OR'	$h_2 = \text{step}(x_1 + x_2 - 1.5)$ 'AND'	$y = h_1 - h_2$
(0, 0)	0	0	0 — silent
(1, 0)	1	0	1 — FIRE
(0, 1)	1	0	1 — FIRE
(1, 1)	1	1	0 — silent

One neuron draws one line — XOR needs a bend. Layer 1 builds OR and AND; layer 2 subtracts them (full version: problem set).

3

PART 3 · LEARNING

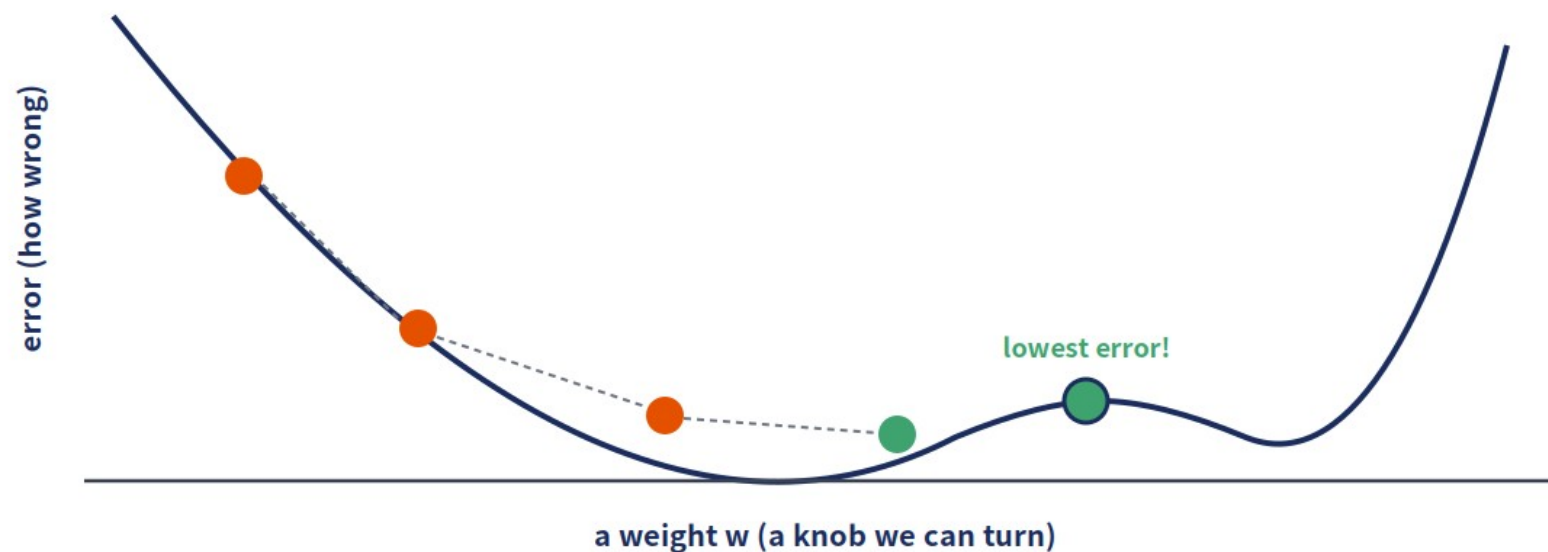
How Networks Learn

Gradient descent & backpropagation

Gradient Descent — Walk Downhill

Feel which way is downhill, take a small step, repeat — until the error is as low as it gets.

Gradient Descent — Walk Downhill in the Fog



Blindfolded hiker

You can't see the whole valley (it's foggy).

But you can feel which way is downhill, and take a small step that way.

Repeat until you reach the bottom (least error).

The Learning Step, as a Formula

Nudge each weight a little, in the downhill direction

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Reading it

- w on the left is the new weight; w on the right is the old one.
- The fraction is the gradient: which way (and how steeply) error changes.
- η (the learning rate) is how big a step we take.

Getting the step right

- Too big: you overshoot the valley and bounce around.
- Too small: training crawls forever.
- η is a key knob — the cousin of C and λ from earlier weeks.

Walking Downhill by Hand

Error $E(w) = (w - 3)^2$, start at $w = 0$ — three step sizes, three fates (full version: problem set)

η	w path from 0	Gap to the valley ($3 - w$)	Fate
0.25	$0 \rightarrow 1.5 \rightarrow 2.25 \rightarrow 2.625$	$3 \rightarrow 1.5 \rightarrow 0.75 \rightarrow 0.375$	halves each step — steady
0.5	$0 \rightarrow 3.0$ (done)	$3 \rightarrow 0$	one perfect step (toy luck)
1.1	$0 \rightarrow 6.6 \rightarrow -1.32 \rightarrow 8.18$	$3 \rightarrow -3.6 \rightarrow 4.32 \rightarrow -5.18$	overshoots, bounces, DIVERGES

On this parabola the gap scales by $(1 - 2\eta)$ each step — the whole 'learning-rate' drama in one factor.

Backpropagation — Pass the Blame Backward

How a deep network knows which knob to turn

A relay team loses a race. To improve, each runner needs to know how much THEY contributed to the loss. Backprop sends the error backward through the layers, telling each weight how to adjust.

The two-step rhythm

- Forward: data flows in, a prediction comes out.
- Backward: the error flows back, blaming each weight.
- Then gradient descent nudges every weight.

Why it was the breakthrough

- It made training MANY layers possible.
- Without it, deep networks couldn't learn.
- You won't compute it by hand — libraries do it for you.

When to Reach for a Neural Net

An honest map — deep nets are a tool, not a religion

Your data looks like...	Reach for	Why
A small, noisy tabular file (most finance)	XGBoost / linear (W4, W7)	Sample-efficient, robust, explainable
Text: news, filings, chat logs	Neural nets (→ LLMs, W12)	Language lives in learned representations
Images: documents, signatures, charts	Deep nets (CNNs)	Layers build edges → shapes → objects
Rich 'normal' to fence for anomalies	Autoencoder (today, #4)	Non-linear essence of normal

The bitter truth of Week 1 still rules: on thin noisy tables, simpler usually generalizes better.

4

PART 4 · AUTOENCODER

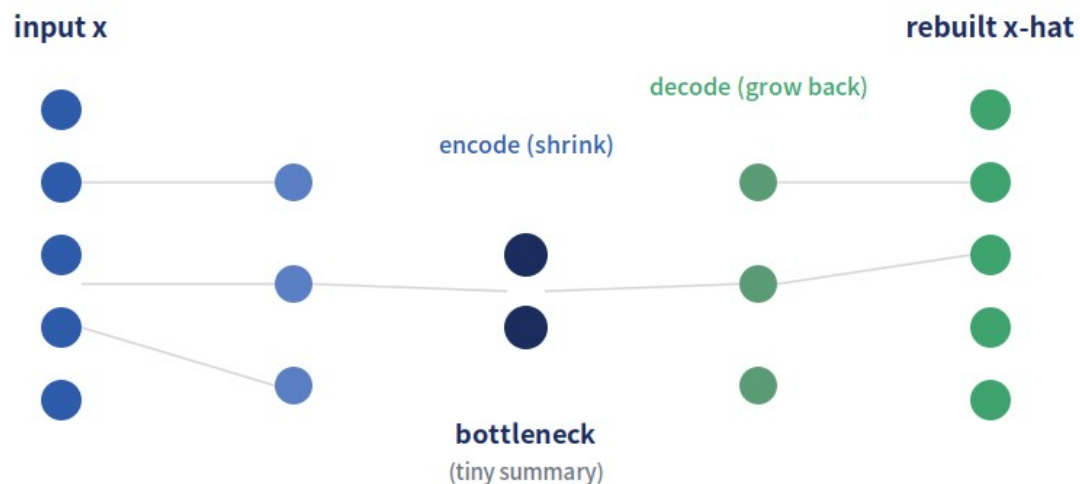
The Bottleneck Detective

Autoencoder — anomaly detector #4

Autoencoder — Squeeze, Then Rebuild

A network that learns to copy its input through a tiny bottleneck. Trained on normal data, it fumbles the weird ones.

Autoencoder — Squeeze, Then Rebuild



Anomaly detector #4

Train it **ONLY** on normal data, forcing it through the narrow bottleneck.

It learns to rebuild normal points well (small error).

Show it a weird point and it rebuilds it badly (big error) → anomaly!

Like a copier trained only on normal forms.

Spotting Anomalies by Rebuild Error

If the rebuild is bad, the point was strange

$$\text{error} = \|x - \hat{x}\|^2 \quad (\text{big} = \text{anomaly})$$

Why it works

- The bottleneck forces it to keep only the essence of normal data.
- Normal points fit that essence → rebuilt accurately.
- Anomalies don't fit → rebuilt poorly → big error.

Strengths

- Captures complex, non-linear 'normal' shapes.
- No labels needed — train on normal only.
- Great for rich data (images, sequences, logs) — the deep-learning member of the family.

A Two-Pixel Autoencoder by Hand

Normal spending is balanced: (food, travel) near the diagonal. The bottleneck keeps ONE number: the average

Point (food, travel)	$z = (x_1 + x_2)/2$	Rebuild $\hat{x} = (z, z)$	Rebuild error	Verdict
(4, 4) — balanced	4	(4, 4)	0	normal
(6, 6) — balanced, bigger	6	(6, 6)	0	normal
(8, 2) — all food, no travel	5	(5, 5)	$(8-5)^2 + (2-5)^2 = 18$	ANOMALY

The bottleneck memorized the diagonal — the 'essence' of normal. Whatever lives off it cannot be rebuilt (full version: problem set).

The Full Anomaly-Detector Family

Four detectors, four different ways to spot 'odd' — we've now met them all

Detector	Core idea	Best when	Type
LOF	Low local density (a loner)	Mixed dense & sparse	distance
OCSVM	Fence around normal	High-dimensional, stable	boundary
Isolation Forest	Few cuts to isolate	Large data, very fast	tree
Autoencoder	Hard to rebuild (big error)	Complex, non-linear	neural net

In practice, banks run several together — each catches what the others miss.

The Bottleneck Is a Dial Too

How much essence to keep — the U-curve, detector edition

Bottleneck too WIDE

- The network can copy everything — a photocopier.
- Even anomalies rebuild perfectly; errors are all ~ 0 .
- The detector goes blind: nothing looks odd.

Bottleneck too NARROW

- Not even normal points fit through.
- Everything rebuilds badly; the alarm never stops.
- Rule of thumb: squeeze until normal JUST fits — validate the squeeze.

Same U-curve as h , λ , k , C , γ , v and η — one idea, eight costumes so far.

Where Neural Nets Fit Our Three Apps

One toolkit, three jobs across the course

Fraud detection

Autoencoder

Learns normal spending; weird transactions rebuild badly → flagged. Detector #4 joins the family.

completes the toolkit

AI chatbot

→ LLMs (Week 12)

Neural nets read language as numbers — the foundation of the LLMs that will power our chatbot.

from words to replies

Recommendation

preference nets

Deep nets blend many signals into one preference — the 'you may also like' engine.

personalization at scale

Neural nets are general-purpose — but use them where data is plentiful, not on thin noisy tables.

Which Piece of the Net? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Which piece?
A single yes/no score from weighted clues	One neuron — Week 5's logistic in disguise
'Fire on exactly one clue' breaks your single line	Two layers — the XOR lesson; depth adds bends
Training loss bounces wildly and explodes	η too big — overshooting the valley
Ten layers but no activation between them	Still one straight line — add the switch (ReLU)
No labels, complex 'normal', need detector #4	Autoencoder — flag the big rebuild errors

If you got 4 of 5, the machinery of Week 8 is yours.

5

PART 5 · HANDS ON

Your Turn — Hands On

Train a small net and complete the four-detector bake-off

Getting This Week's Data

Companion notebook — reuse the § 5 classification set

From the companion notebook (§ 5)

- Run Setup (§ 0), then the § 5 offline cell.
- Gives X (clues) and yb (label, ~1% positive).
- Same data as Weeks 5–7 — compare against earlier models.

What we'll do

- Lab 1: a small neural net (MLP) vs earlier models.
- Lab 2: an autoencoder for anomaly detection.
- Both in scikit-learn — no GPU needed (real AEs use Keras).

[CODE] FML_Data_Setup_Guide.ipynb · § 5 offline classification set

```
from sklearn.datasets import make_classification
X, yb = make_classification(n_samples=20000, n_features=20, n_informative=6,
                           weights=[0.99, 0.01], random_state=42)
# X = clues, yb = label (about 1% positive, fraud-like)
```

Hands-On Lab 1 — A Small Neural Network

Train an MLP and compare it to last week's models

[CODE] colab_lab1.py · data from setup guide § 5

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score

Xs = StandardScaler().fit_transform(X) # nets like scaled data
Xtr, Xte, ytr, yte = train_test_split(Xs, yb, random_state=0)

net = MLPClassifier(hidden_layer_sizes=(32, 16),
                    activation='relu', max_iter=300)
net.fit(Xtr, ytr)
print('MLP F1:', round(f1_score(yte, net.predict(Xte)), 3))
```

Your task (step by step)

- Load X, yb (setup guide § 5).
- Scale the features (nets like tidy inputs).
- Train an MLP with two hidden layers.
- Compare its F1 to Forest / Boosting.
- Q: does the neural net win on this small table?

Hands-On Lab 2 — Autoencoder Anomalies

Rebuild the data; big rebuild error = anomaly

[CODE] colab_lab2.py · data from setup guide § 5

```
import numpy as np
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

Xs = StandardScaler().fit_transform(X)
# autoencoder: rebuild the input through a bottleneck
ae = MLPRegressor(hidden_layer_sizes=(8,), max_iter=300)
ae.fit(Xs, Xs)

err = np.mean((Xs - ae.predict(Xs))**2, axis=1) # rebuild error
thresh = np.quantile(err, 0.99)
is_anom = err > thresh
print('caught', int(np.sum(is_anom & (yb == 1))),
      'real anomalies')
```

Your task (step by step)

- Train an MLP to output its OWN input (X to X).
- Use a small hidden layer (the bottleneck).
- Measure rebuild error per point.
- Flag the top ~1%; check vs yb.
- Q: how does it compare to the other 3 detectors?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 and Lab 2 code above, one plain-English sentence per line.
- 2) Sweep the autoencoder bottleneck size over [2, 4, 8, 16, 20]; plot mean rebuild error of normal vs anomaly rows per size - where does the gap between the two curves peak?
- 3) Run the four-detector bake-off on the same 8000 rows with a 1% flag budget each: LOF, OCSVM, IsolationForest, Autoencoder - table of catches.
- 4) Add a majority-vote (2-of-4) row to the table.
- 5) In 3 sentences: which detector caught what the others missed, and why the bottleneck sweep is the U-curve again.

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 Why do neural networks need an activation function?

- A** To speed up training
- B** To add non-linearity for curvy patterns ✓
- C** To delete features
- D** To label the data

Answer: Without a non-linear switch, stacking layers just gives one straight line — no extra power.

Q2 How does an autoencoder flag an anomaly?

- A** It has a label for it
- B** It rebuilds the point badly (big error) ✓
- C** It is the fastest model
- D** It counts neighbors

Answer: Trained on normal data, it rebuilds normal points well; weird points get a big rebuild error.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 Training loss keeps bouncing higher and higher. Most likely:

- A** The bottleneck is too narrow
- B** η is too big — steps overshoot the valley ✓
- C** Too few features
- D** The data is too clean

Answer: Our hand-walk showed it: at $\eta = 1.1$ the gap grows $\times 1.2$ each step. Shrink the learning rate.

Q4 Your autoencoder's rebuild errors are ~ 0 for EVERYTHING, anomalies included:

- A** Perfect — deploy it
- B** Bottleneck too wide — a photocopier ✓
- C** η is too small
- D** The labels leaked

Answer: With too much capacity it copies anomalies too — squeeze the bottleneck until only 'normal' fits.

Week 8 on One Page

Photograph this slide — it is the whole lecture

Concept	One line to remember
Neuron	weighted sum + fire switch; with a sigmoid it IS logistic (W5).
Depth	layers stack bends — XOR falls to two layers, never to one.
ReLU	pass positives, flatten negatives — the cheap bend that works.
Gradient descent	step downhill by η ; too big bounces, too small crawls.
Backprop	the error walks backward, blaming each weight on the way.
Autoencoder	rebuild through a bottleneck; big error = anomaly (#4 of 4).

Next stop: unsupervised learning — structure with no labels at all.

Four Ideas, One Picture Each

Bonus — Autoencoder: rebuild through a bottleneck; a big rebuild error flags an anomaly (detector #4)

1

A neuron weighs clues and fires

Weighted sum + an activation switch. One neuron = logistic regression; power comes from stacking.

2

Deep = many layers, simple to complex

Early layers catch edges, later ones catch whole ideas. Hungry for data — careful in finance.

3

Networks learn by walking downhill

Backprop passes the blame backward; gradient descent nudges each weight to shrink the error.

Keep the pictures: a weigh-and-fire brick, stacked bends, a downhill walk — and a bottleneck detective.

Week 8 — Three Things to Remember

Before next class: finish Labs 1–3 and compare all four anomaly detectors on the same data

1

The neuron is the building block

Weigh inputs, add a bias, pass through an activation. Stack millions and you get modern AI.

2

Deep nets learn features and patterns

Layers build simple to complex; they learn via gradient descent + backpropagation. Data-hungry.

3

Autoencoder completes our 4 detectors

Rebuild through a bottleneck; big error = anomaly. LOF, OCSVM, Isolation Forest, Autoencoder.

Next week — Unsupervised Learning: clustering, PCA, and denoising the covariance matrix.